

FlowWatch

Netværksangrebsdetektion med Machine Learning

10 min præsentation · 15 min eksaminering

Machine Learning · DATA-GBG-E24-V-MAL · EAK 2026

Problem	Klassificér netværkstrafik som BENIGN eller én af 14 angrebstyper i realtid
Datasæt	CICIDS2017 — 2,52M flows, 78 features, 15 klasser, kraftig class imbalance
Notebooks	preprocess.ipynb → xgboost_ids.ipynb → dnn_comparison.ipynb
Resultater	XGBoost: Accuracy 99,55% F1 0,7981 ROC AUC 0,9999 · DNN: Accuracy 9,68% F1 0,0523
Deployment	ONNX-model i Node.js/Express API — demo.sh tester live mod rigtige flows

Del 1 — Præsentation (10 minutter)

Tidsbadgen viser hvornår du skal være nået til punktet. Kursiv er forslag til hvad du siger — lær dem ikke udenad.

0:00

Hvad er FlowWatch?

» *Mit projekt hedder FlowWatch. Det er et system der i realtid kan kigge på netværkstrafik og afgøre om det er normalt eller et angreb — og i så fald hvilken type angreb.*

» *Problemet jeg løser er relevant fordi netværksangreb sker konstant, og manuel overvågning ikke skalerer. Med machine learning kan man automatisere det.*

» *Jeg bruger datasættet CICIDS2017 som er et industri-benchmark skabt specifikt til at træne og teste netværkssikkerhedsmodeller. Det indeholder 78 tekniske features per flow og 15 klasser — BENIGN og 14 angrebstyper som DDoS, PortScan, Bot og SQL Injection.*

Vis: Overblik over klasserne og datasættet.

1:00

Dataforbehandling — preprocess.ipynb

» *Inden jeg kan træne en model skal data renses. Jeg starter med at indlæse 8 separate CSV-filer fra CICIDS2017 og merger dem til ét samlet datasæt med 2,83 millioner rækker.*

» *Derefter går jeg igennem fire rensningsskridt. Først fjerner jeg duplikerede rækker*

— det bringer datasættet ned på 2,52 millioner rækker. Derefter dropper jeg uendelige værdier, som opstår fordi nogle features beregnes ved at dividere med flow-varighed, og hvis den er nul får man uendelig. Dem kan en model ikke håndtere. Derefter fjerner jeg NaN-værdier.

» Til sidst label-encoder jeg klasse-navnene — altså konverterer tekst som 'DDoS' og 'BENIGN' til integers fra 0 til 14 — fordi modeller arbejder med tal, ikke tekst.

» Resultatet gemmes som `cleaned.csv` og bruges som startpunkt af begge modeller.

Vis: Rækketallene og label-mappingen i notebook-outputtet.

2:00

XGBoost + Optuna — `xgboost_ids.ipynb`

» Den største udfordring i dette datasæt er *class imbalance*. Langt de fleste flows er BENIGN, og sjældne angrebstyper som Heartbleed har kun 2 eksempler i hele testsættet. Det betyder at en naiv model bare kan sige BENIGN for alt og opnå høj accuracy — men den er fuldstændig ubrugelig i praksis.

» Jeg løser det ved at beregne *sample weights*. Sjældne klasser får en højere vægt under træning, så modellen ikke kan ignorere dem. Formlen er: $\text{weight} = \text{total antal eksempler} / (\text{antal klasser} \times \text{antal eksempler i klassen})$.

» Til selve modellen starter jeg med en baseline XGBoost og tuner derefter hyperparametre med Optuna. Optuna er Bayesian optimering — i stedet for at prøve alle kombinationer blindt som grid search, lærer det af tidligere forsøg og foreslår intelligente næste værdier. Jeg kørte 50 forsøg på 15% af træningsdata.

» Den endelige model trænes med de bedste parametre på alt data — 300 træer — og opnår Accuracy 99,55%, F1 macro 0,7981 og ROC AUC 0,9999.

Vis: Optuna-resultater og forbedringen fra baseline til optimeret model.

4:00

Resultater og grafer

» Nu vil jeg vise hvad modellen faktisk kan. Jeg har tre visualiseringer der hver fortæller noget forskelligt.

» Confusion matrix viser for hver af de 15 klasser hvor mange flows der blev klassificeret korrekt og hvad de forkerte blev forvekslet med. Diagonalen skal helst være mørkeblå — og det er den.

» Feature importance viser hvilke af de 78 features modellen lagde mest vægt på. Det er interessant fordi det giver os indblik i hvad der faktisk adskiller angrebstrafik fra normal trafik.

» t-SNE er en visualisering der tager de 78 features og projicerer dem ned til 3 dimensioner, så vi kan se om klasserne naturligt separerer sig. Det er en god sanity check på om data overhovedet er klassificerbart.

Vis: Peg konkret på hver graf mens du taler — se næste sektion for hvad du siger.

8:00

DNN-sammenligning — `dnn_comparison.ipynb`

» Den tredje notebook træner et Deep Neural Network på samme data. Formålet er ikke at slå XGBoost — det er at demonstrere at jeg forstår forskellen på de to tilganges fundamentale måde at lære på.

» XGBoost bygger beslutningstræer sekventielt. Hvert træ er en if-else struktur der splitter data på feature-værdier — der er ingen neuroner, ingen vægte der opdateres, ingen aktiveringsfunktioner.

» DNN'et derimod lærer ved at sende data fremad gennem lag af neuroner, beregne fejlen, og sende den baglæns for at justere alle vægtene. Det er en helt anden læringsmekanisme.

» Resultatet er som forventet: XGBoost vinder med F1 0,7981 mod DNN'ets 0,0523. Men det er ikke en fair sammenligning — XGBoost er tunet med 50 Optuna-forsøg og 300 træer, DNN'et kørte 20 epochs uden nogen tuning.

Vis: Training curves og sammenligningsgraferne.

9:00

Deployment — ONNX og Node.js

» Det sidste trin er at gøre modellen tilgængelig i produktion. Jeg eksporterer XGBoost-modellen til ONNX-format, som er et åbent standard der lader mig bruge den trænede model i Node.js uden at skulle genimplementere logikken.

» API'et er bygget i Node.js/Express med fire endpoints: et health check, et features-endpoint der viser hvilke inputs modellen forventer, og to predict-endpoints — ét til enkelt JSON-input og ét til batch-klassifikation fra CSV.

» Lad mig køre demo.sh som tester modellen live mod rigtige flows fra datasættet.

Vis: demo.sh — benign → BENIGN, DDoS → DDoS, PortScan → PortScan.

► Hold øjenkontakt og peg aktivt på skærmen. Censor vil se at du forstår hvad der sker — ikke at du læser op.

! Eksaminatoren må godt afbryde. Svar kortfattet og tilbyd at uddybe bagefter.

Graferne — sådan præsenterer du dem

Confusion Matrix

Faktisk klasse (y-akse) vs. forudsagt klasse (x-akse). Tallene på diagonalen er korrekte forudsigelser.

Det siger du: *Diagonalen er mørkeblå — næsten alt er korrekt. Klasse 0 er BENIGN med over 416.000 rigtige. Klasse 8 er Heartbleed — kun 2 eksempler i testsættet, begge klassificeres korrekt. Største fejlkilde: klasse 14 Web XSS forveksles lidt med klasse 12 — de er strukturelt ens angreb.*

Feature Importance — Top 20

Hvilke af de 78 features XGBoost brugte mest til sine beslutninger.

Det siger du: *Klart vigtigst er Bwd Packet Length Max — maksimal pakkellængde i bagudrettet trafik. Angrebstrafik som DDoS sender ensartede pakker af en bestemt størrelse, normal trafik er mere varieret. God validering af at modellen har lært noget meningsfuldt og ikke bare overfit.*

3D t-SNE

78 features projiceret ned til 3 dimensioner — viser om klasserne naturligt separerer sig.

Det siger du: *Klasserne separerer sig pænt selv med 78 features komprimeret til 3D — det forklarer den høje performance. De lange strenge er DoS-angreb der genererer mange ensartede flows. Heartbleed og Infiltration er små kompakte klynger fordi de er sjældne og homogene.*

Del 2 — Eksaminering (15 minutter)

DNN-pensum

Sp: Hvad er en Deep Neural Network?

Sv: Det er et system af lag der er inspireret af hjernen. Data sendes ind i den ene ende, beregnes igennem en række lag, og et svar kommer ud i den anden ende. Jo flere lag, jo mere komplekse mønstre kan systemet lære at genkende. Det kaldes deep fordi der er mange lag.

Sp: Hvad er input?

Sv: Input er de rå data man giver netværket. I mit projekt er det 78 tal der beskriver et netværksflow — fx pakkelængder, antal pakker og flow-varighed. Hvert tal svarer til én indgang i netværket.

Sp: Hvad er output?

Sv: Output er det svar netværket giver tilbage. I mit projekt er det en sandsynlighedsscore for hver af de 15 klasser — den klasse med den højeste score er netværkets bud på hvad trafikken er.

Sp: Hvad er vægte?

Sv: Vægte er de tal der styrer hvor meget indflydelse hvert input har på det næste lag. Når et netværk træner, er det præcis disse tal det justerer — igen og igen — indtil det bliver godt til at genkende mønstre.

Sp: Hvad er bias?

Sv: Bias er et ekstra tal i hver beregningsenhed der giver den mulighed for at forskyde sit output uafhængigt af hvad den modtager som input. Det giver netværket mere fleksibilitet til at tilpasse sig data.

Sp: Hvad er et hidden layer?

Sv: Et hidden layer er et mellemlag mellem input og output. Det er her netværket finder mønstre i data. De tidlige lag finder simple sammenhænge, og de dybere lag kombinerer dem til mere komplekse forståelser. Hidden betyder blot at det er skjult — man ser ikke direkte hvad der sker der.

Sp: Hvad er forward propagation?

Sv: Det er den proces hvor data bevæger sig fremad igennem netværket fra input til output. Hvert lag tager det forrige lags output, beregner noget med det, og sender resultatet videre til næste lag. Til sidst ender man med et svar.

Sp: Hvad er summerings-funktionen (Sigma)?

Sv: Det er den beregning der sker i hver enhed i netværket. Den tager alle de tal den modtager, ganger hvert af dem med dets tilhørende vægt, lægger dem alle sammen, og tilføjer et ekstra tal. Resultatet er et enkelt tal der sendes videre.

Sp: Hvad er en aktiveringsfunktion?

Sv: Det er en funktion der bestemmer hvad en enhed i netværket sender videre baseret på det den har beregnet. Uden den ville hele netværket opføre sig som én simpel lineær beregning uanset hvor mange lag det har — aktiveringsfunktionen er det der giver netværket evnen til at lære komplekse mønstre.

Sp: Findes der forskellige aktiveringsfunktioner?

Sv: Ja, der er flere. Nogle sætter negative værdier til nul og lader positive værdier passere uændret. Andre klemmer alle værdier ind i et interval mellem nul og et, så outputtet kan tolkes som en sandsynlighed. I mit projekt bruges en type der fordeler outputtet over alle klasser så de summer til hundrede procent.

Sp: Hvad er target?

Sv: Target er det rigtige svar for et givent eksempel i træningsdata. Det er det netværket forsøger at ramme. Under træning sammenlignes netværkets svar med target for at måle hvor forkert det var.

Sp: Hvad er error?

Sv: Error er et mål for hvor meget netværkets svar afviger fra det rigtige svar. Jo lavere error, jo bedre klarer netværket sig. Hele formålet med træning er at minimere denne fejl over tid.

Sp: Hvad er backpropagation?

Sv: Backpropagation er den proces der sker efter netværket har givet et svar og vi har beregnet fejlen. Fejlen sendes baglæns igennem netværket lag for lag, og undervejs beregnes det hvor meget hvert enkelt tal i netværket bidrog til fejlen. Den information bruges til at justere tallene så netværket klarer sig bedre næste gang.

Sp: Hvad er en gradient?

Sv: En gradient siger noget om i hvilken retning og med hvor meget kraft et bestemt tal i netværket skal justeres for at reducere fejlen. Hvis gradienten er positiv skal tallet sænkes, er den negativ skal det hæves.

Sp: Hvad er learning rate?

Sv: Learning rate bestemmer hvor store skridt netværket tager når det justerer sine tal. Tager det for store skridt risikerer det at springe over den bedste løsning. Tager det for små skridt tager træningen meget lang tid.

Sp: Hvad er momentum?

Sv: Momentum er en mekanisme der giver justeringerne en form for hukommelse. I stedet for at justere udelukkende baseret på den aktuelle situation, tages der også hensyn til hvilken retning justeringerne har bevæget sig i de foregående skridt. Det gør træningen mere stabil og hjælper med at undgå at sidde fast.

Sp: Hvordan udregnes en ny vægt?

Sv: Man tager den nuværende vægt og trækker et lille skridt fra baseret på gradienten. Skridtstørrelsen styres af learning rate. Hvis man bruger momentum blandes den aktuelle retning med historikken fra tidligere justeringer, inden man trækker det fra vægten.

Sp: Hvad er en Decision Tree?

Sv: En decision tree er en model der lærer at forudsige ved at stille en serie spørgsmål om data — lidt som et flowchart. For hvert spørgsmål splittes data i to grupper, og processen gentages

indtil man når et endeligt svar. Den er nem at forstå og forklare, men kan have svært ved at generalisere til nye data.

Sp: Hvad er XGBoost?

Sv: XGBoost er en metode der bygger mange decision trees oven på hinanden. Hvert nyt træ fokuserer på de eksempler som de tidligere træer tog fejl på. På den måde forbedrer systemet sig trinvist. Det er meget effektivt på strukturerede data som tabeller, og det fortæller dig hvilke features der var vigtigst for forudsigelserne.

Projektspecifikt

Sp: Hvad er class imbalance og hvordan håndterede du det?

Sv: Class imbalance betyder at nogle klasser har langt færre eksempler end andre i træningsdata. I mit datasæt er de fleste flows BENIGN — en model kan derfor opnå høj træningsnøjagtighed ved blot at sige BENIGN for alt, uden nogensinde at lære at genkende angreb. Jeg løser det ved at give sjældne klasser højere vægt under træning, så modellen straffes hårdere for at tage fejl på dem.

Sp: Hvad lavede du i preprocessing?

Sv: Jeg indlæste og mergede 8 CSV-filer til ét datasæt. Derefter fjernede jeg duplikerede rækker, droppede ugyldige værdier der opstod ved division med nul, og fjernede rækker med manglende værdier. Til sidst konverterede jeg klasse-navnene fra tekst til tal så modellen kan arbejde med dem.

Sp: Hvad er Optuna og hvorfor brugte du det?

Sv: Optuna er et værktøj til automatisk at finde de bedste indstillinger for en model. I stedet for manuelt at prøve forskellige kombinationer bruger det tidligere forsøgs resultater til at foreslå hvad der er værd at prøve næste gang. Det sparer meget tid sammenlignet med at prøve alle kombinationer systematisk.

Sp: Hvad er ONNX og hvorfor brugte du det?

Sv: ONNX er et filformat til at gemme trænede modeller på en måde der er uafhængig af det programmeringssprog eller framework de blev trænet i. Jeg brugte det for at kunne tage min model trænet i Python og bruge den direkte i mit Node.js API uden at skulle genimplementere noget.

Sp: Hvad er den fundamentale forskel på DNN og XGBoost?

Sv: XGBoost lærer ved at bygge en serie af simple regler baseret på feature-værdier — det er ikke inspireret af hjernen og bruger ikke neuroner eller vægte. Et DNN derimod er opbygget af lag af små beregningsenheder der hver især justerer deres interne tal baseret på fejlen fra det forrige forsøg. De to metoder bruger fundamentalt forskellig matematik og logik til at nå det samme mål.

► Du har 25 minutter. Er der 5 minutter tilbage og du ikke er nået til deployment — spring det over.

! Graferne er dit stærkeste kort. Brug dem aktivt og peg på konkrete tal.